AI Report for Lab 3

(1) Prompt to AI for 3.3.1:
"hi chat, please give me a recommendation for a technology stack for my medium-lightweight mobile application. my application is a route recommendation system that generates routes for users based on their preferences and needs. the generated routes take into consideration accessibility needs for disabled persons and shaded routes for users who prefer so as well. we use LTA's OneMap API to feed in the live information for the route planning and the app generates a list of routes with the route that satisfies the user's request the best at the top. please generate a technology stack for this application. a constraint is my team prefers to code in python for the backend. give me a recommendation for the frontend, backend and database. ive attached our class diagram so that you have a better idea of what functionalities are required for our app. i also want to specify that we have decided to use MongoDB as it is faster for an MVP and we prefer to use a non-relational database for dynamic JSON objects."

Attached complete class diagram as well for input to AI.

AI's output to 3.3.1:

Final recommendation

Here is the stack I would commit to for your team:

Layer	Final choice
Mobile framework	**React Native + Expo**
Frontend language	**TypeScript**
Maps	`react-native-maps`, subject to validating OneMap integration
Backend framework	**FastAPI**
Backend language	**Python**
Request and response validation	Pydantic
Database	**MongoDB Atlas**
Database driver	PyMongo
External API integration	HTTPX + OneMap API
Route-ranking logic	Custom Python scoring service

Authentication	Managed identity provider or securely implemented FastAPI authentication
Testing	pytest + FastAPI TestClient
Source control	GitHub
Deployment	Managed Python hosting + MongoDB Atlas

My three most important recommendations

1. Do not overcomplicate the infrastructure. A modular FastAPI backend, MongoDB Atlas, and a single React Native application are sufficient for an MVP.

2. Validate your mapping approach early. The main integration risk is not whether Python can call OneMap; it is whether your selected map-rendering approach, route data, accessibility data, and shade data can work together to deliver the experience you envision.

3. Prioritise the route-ranking engine. React Native, FastAPI, and MongoDB are established technologies. Your application's real value lies in how accurately and transparently it evaluates routes against users' accessibility needs and preferences.

In short: React Native + Expo, FastAPI + Python, and MongoDB Atlas is the stack I recommend. It matches your team's preferences, supports your class diagram, and gives you a practical path from an initial prototype to a more capable route recommendation application.

Prompt to AI for 3.3.2:

In my previous exercise I asked for a technology stack recommendation and you recommended the following:

Recommended stack

Layer	Recommendation	Main technologies
Frontend	React Native	**React Native + Expo + TypeScript + MapLibre**
Backend	Python REST API	**FastAPI + Pydantic + HTTPX + SQLAlchemy**

Database	Relational + geospatial DB	**PostgreSQL + PostGIS**
External data	Singapore map/routing	**OneMap APIs**
Deployment later	Managed hosting	Render/Railway/Fly.io + managed PostgreSQL

For this project we have now decided to use Python with FastAPI and MongoDB instead of PostgreSQL with PostGIS and SQLAlchemy. Please compare our original recommendation with our current choice for our route recommendation app and tell me whether you agree or disagree with our decision to change.

Answer to 3.3.2:
The two recommendations are mostly the same with both choosing Python and FastAPI, React Native with TypeScript and Expo, Pydantic, HTTPX and the OneMap API. The main difference is the database where Lab 2 recommended PostgreSQL with PostGIS and SQLAlchemy, while Lab 3 recommended MongoDB Atlas with PyMongo. Lab 3 also replaces MapLibre with react-native-maps and adds libraries for navigation, forms and location.

(1) The database change comes directly from our prompt in which we chose to specify that we decided to use MongoDB. The other additions may come from our more developed Lab 3 class diagram, which adds screens, location and hazard classes and route scoring strategies. The map library change is not explained by the database, since the AI said it would not switch libraries for that reason, so it may be variation between runs.

(2) We asked the AI to compare its Lab 2 recommendation with our choice of MongoDB. It initially partially disagreed arguing PostGIS is stronger for geospatial analysis, but concluded that it conditionally agrees with MongoDB for an MVP if route scoring is done in Python. It also said MongoDB is not automatically faster, and that PostgreSQL handles flexible JSON too.

(2) Prompt to AI for 3.3.3:
“hi chat, so here is the completed class diagram and our preferred choice of programming language is python. using this info, generate a minimal skeleton template for my team's project, consisting of a folder hierarchy containing files that

are empty except for TODO comments. do not put any code inside the files! give me a zip folder as an output."

Attached complete class diagram as well for input to AI.

AI's output to 3.3.3:
I've created your project skeleton as a ZIP folder, based on your class diagram and Python preference.

walkingsg_project_skeleton.zip

What's included?

The ZIP contains **84 files**, organised into a Python backend project structure.

```
routewise_project_skeleton/
├── app/
│   ├── api/            # API endpoints
│   ├── controllers/    # Controllers from your class diagram
│   ├── core/           # Configuration and security
│   ├── db/             # Database setup
│   ├── integrations/   # OneMap and email integrations
│   ├── models/         # User, Route, Hazard, Profile, etc.
│   ├── repositories/   # Data access
│   ├── schemas/        # API request/response schemas
│   ├── services/       # Business logic
│   ├── strategies/     # Route-scoring strategies
│   └── main.py
├── alembic/
├── tests/
├── .env.example
├── .gitignore
├── Dockerfile
├── docker-compose.yml
├── requirements.txt
└── README.md
```

What I've kept in mind

- **Your class diagram:** Includes the main controllers, entities, repositories, and route-scoring strategies.
- **Python preference:** The skeleton is designed around FastAPI and Python.
- **Minimal implementation:** Every file contains only TODO comments—no executable code or implemented functionality.
- **Future development:** The structure provides dedicated places for OneMap integration, accessibility and shade scoring, hazard management, authentication, and database operations.

Note: This ZIP contains the Python backend skeleton, not a complete mobile frontend. Your team can develop the mobile frontend separately and connect it to the backend through its API.

Answer to 3.3.4:
The AI followed the instruction with every file contains a single TODO comment and no code and we agree with the layered structure of controllers, services, repositories, models, the strategy package for route scoring, a repository base, and separate integration modules for the map service. It also produced all eight control classes from the diagram.

However, it did get some items wrong. It produced no boundary classes or mobile folder, even though the product is a mobile app. It omitted the OTP entity, the SessionManager, and the Observer and Singleton elements. Its scoring strategies which include shade, accessibility and shortest time, do not match the diagram's scoring criteria consisting of shortest distance, shelter priority and whether a route is stair-free, and some TODOs hedge on things the diagram states clearly. The TODOs are also generic and do not refer to the diagram's attributes or operations. We also disagree with its deployment choices as we want to focus on producing features. Hence, we decided not to use Docker as our demo does not need deployment and DevOps tooling.

It did not add any implementation code, but it did add files and folders beyond the diagram such as Docker files, Alembic, requirements, tests, a 'schemas' folder, an API folder and a service layer. It also assumed PostgreSQL with SQLAlchemy, which we did not specify and mentioned hazard "moderation" which is not in our design.